When adding an Enumerated Type to your database it forms part of the database structure.

You may be familiar with Enums in Rails. In Active Record, an Array of Enum values can be used with an inclusion validation to ensure input data matches one of the types.

Although not an Enumerated Type specifically, the VehicleStatus class in the Rideshare codebase limits possible status options to Strings that are DRAFT or PUBLISHED.

Rideshare uses an Inclusion validation in the Vehicle model to constrain the possible values for status.

```
validates :status, inclusion: { in: VehicleStatus::VALID_STATUSES }
```

The status column also uses a NOT NULL constraint and a database column default of the String returned by VehicleStatus::DRAFT.

In the next section, you'll see how to use database enumerated types to constrain the input data.

The Rails Guides page on Active Record and PostgreSQL covers Enumerated Types.[14]

- Enums are a PostgreSQL type
- Additional enum values can be added either before or after existing values, but not in the middle
- Enum values cannot be dropped

The position of new values, and the inability to drop existing values, are significant trade-offs to be aware of.

If you require more flexibility than that, you may wish to leave the enumerated type in application code. If you do not imagine the values changing much, then putting them in the database provides stronger consistency guarantees at the cost of more rigidity.

First, the enum is added using the create_enum Ruby method. Second, the column is added and the type is set using the enum name. The default value and null handling are the same.

Copy and paste the following code into the change method in the Migration file you generated, and then run bin/rails db:migrate to apply it.

14. https://guides.rubyonrails.org/active_record_postgresql.html#enumerated-types